Lab 3 — Complete Study Guide

Linear Regression, Logistic Regression & Regularization

Every Topic Explained In Depth

Prepared for study and exam review

Part 1: What is Linear Regression?

Core Concept

Linear regression is the simplest supervised machine learning model for regression tasks — predicting a continuous numeric value. It assumes a straight-line (or hyperplane) relationship between inputs and output.

The Mathematical Equation

The model learns the equation: `y = β0 + β1·x1 + β2·x2 + … + βn·xn + ε`

Symbol	Name	Meaning
y	Target variable	The value we predict (e.g., house price)
x1, x2, … xn	Features	The input variables fed into the model
β0	Intercept (bias)	Baseline y when all features are zero
β1 … βn	Coefficients (weights)	How much each feature moves the prediction — learned during training
ε	Residual (error)	The noise the model cannot capture

How Does It Learn? — Ordinary Least Squares

The model finds the best β values by **minimizing the Sum of Squared Residuals (SSR)**:

```
SSR = Σ (y_i − ŷ_i)²
```

Where `ŷ_i` is the prediction and `y_i` is the actual value. This technique is called **Ordinary Least Squares (OLS)**. Scikit-Learn's LinearRegression uses a direct matrix calculation (not gradient descent), so the solution is exact and deterministic — you always get the same answer.

> **TIP:** Think of OLS as finding the line where the total "vertical distance squared" between every point and the line is as small as possible.

Part 2: Loading Dependencies

Before building any model, we import the Python libraries we need. Here is what each one does and why it matters:

Import	Purpose
numpy (np)	Core numerical library. Provides fast N-dimensional arrays and math functions like mean, linspace, random. Nearly every ML workflow depends on NumPy.
pandas (pd)	Tabular data library. DataFrames act like spreadsheets inside Python — you load CSVs, filter rows, create columns, and more.
matplotlib.pyplot (plt)	Foundational plotting library. The pyplot interface gives us scatter plots, line plots, histograms, and full control over labels and axes.
sklearn.datasets	Pre-built datasets bundled with Scikit-Learn (e.g., breast cancer, iris). Great for learning because you skip data collection.
sklearn.preprocessing	Tools for transforming data before training — normalization, standardization, encoding.
sklearn.linear_model	Contains all linear models: LinearRegression, LogisticRegression, Ridge, Lasso, and more.
train_test_split	Utility to randomly split data into training and testing sets with one function call.

TIP: The aliases np, pd, and plt are universal conventions. Using them makes your code readable to any data scientist.

Part 3: Simple Linear Regression (One Feature)

Step 1 — Loading & Inspecting Data

We load a CSV with `pd.read_csv()` and check its shape. The dataset has 100 rows and 2 columns (one feature `x1` and one target `y`). Calling `df.head(10)` shows the first 10 rows so we can visually confirm the data looks reasonable.

Step 2 — Extracting Features & Target

Scikit-Learn requires features as a **2D array** of shape (n_samples, n_features). We use `df[['x1']].values` with double brackets to get a 2D array of shape (100, 1). The target `df['y'].values` is a 1D array of shape (100,).

> **KEY POINT:** Double brackets [['x1']] → DataFrame (2D). Single brackets ['y'] → Series (1D). This distinction is critical for Scikit-Learn compatibility.

Step 3 — Visualization

A scatter plot of x1 vs y reveals whether the data is roughly linear. If points form an upward or downward trend, linear regression is appropriate.

Step 4 — Train/Test Split

We use `train_test_split(X, y, test_size=0.20, random_state=...)` to reserve 20% of data for testing and 80% for training. The `random_state` seed ensures reproducibility — the same split occurs every run.

> **Why Split the Data?**
> If you evaluate a model on the same data it was trained on, you cannot distinguish true learning from memorization. The held-out test set simulates unseen real-world data and gives an honest measure of generalization.

Step 5 — Training & Interpreting Results

Calling `model.fit(X_train, y_train)` trains the model. The learned parameters are:

Intercept (β0) = 1.15 and **Coefficient (β1) = 3.88**

Learned equation: `y ≈ 1.15 + 3.88 × x1`. This means every 1-unit increase in x1 predicts a 3.88-unit increase in y.

Step 6 — Evaluation with MSE

Mean Squared Error (MSE) measures average prediction error: $MSE = (1/n) \times \Sigma(y_i - \hat{y}_i)^2$

Training MSE = 18.03, Testing MSE = 29.16. The test error being higher is normal and expected — the model was optimized on training data, so it naturally performs slightly worse on unseen data.

> **KEY POINT:** A huge gap between training and testing MSE is a red flag for overfitting. A small gap (like here) indicates healthy generalization.

Part 4: Handling Non-Linear Data with Feature Engineering

The Problem

Dataset 2 shows a parabolic (U-shaped) relationship between x and y. A straight line cannot fit a parabola, so plain linear regression would fail badly.

The Solution: Create a New Feature

The key insight is that linear regression is **linear in the parameters** (β values), not in the features. If we add a new column `x²` to the data, the model can learn: `y ≈ β0 + β1·x + β2·x²`. This equation is a parabola, yet the model is still "linear" because β0, β1, β2 are combined linearly.

> **Feature Engineering**
>
> Feature engineering means creating new input columns from existing ones to help the model capture patterns it otherwise could not. Common tricks: squaring, taking logs, multiplying features together, or using domain knowledge.

How to Verify It Worked

Plot `y vs x²` — you should see a linear trend. This confirms the transformed feature has a straight-line relationship with the target, which is exactly what linear regression needs.

Interpreting the Learned Equation

After training with both features [x, x²], the model returns β1 ≈ 0 (the original x has no linear effect) and β2 ≈ 0.51 (x² is the important feature). The equation simplifies to `y ≈ 0.51·x² − 1.70` — exactly the parabolic shape we see in the scatter plot.

Part 5: Multiple Linear Regression (Boston Housing)

The Dataset

The Boston Housing dataset has 506 samples and 13 features describing neighborhoods (crime rate, average rooms, distance to employment centers, etc.). The target is the median home value in thousands of dollars.

Loading Raw Data — Why It Needs Reshaping

The raw file stores each sample across two text lines (11 values on the first line, 3 on the second). We use NumPy array slicing with `[::2, :]` (every even row = first lines) and `[1::2, :3]` (every odd row, first 3 columns = second lines), then `np.hstack` to glue them horizontally into a proper (506, 14) array.

Multiple Features = Hyperplane

With 13 features, the model learns a 13-dimensional hyperplane instead of a simple line. The equation becomes $y = \beta_0 + \beta_1 \cdot x_1 + \beta_2 \cdot x_2 + \ldots + \beta_{13} \cdot x_{13}$. Each coefficient tells you how that feature affects the predicted house price.

Feature Ranges Vary Wildly

Printing the min and max of each column reveals that some features range 0–1, others 0–100, and others 187–711. This disparity is why normalization matters — addressed next.

Part 6: Normalization (MinMax Scaling)

What is MinMax Scaling?

MinMax scaling transforms every feature to the range [0, 1] using: `x_scaled = (x - x_min) / (x_max - x_min)`

After scaling, the smallest value of each feature becomes 0 and the largest becomes 1, with everything else proportionally in between.

Three Reasons to Normalize

Reason	Explanation
1. Gradient-Based Learning (Critical)	Gradient descent updates weights proportionally to feature values. If one feature is 100× larger, its gradient dominates, causing slow or unstable training.
2. Coefficient Interpretability	After normalizing, all coefficients are on the same scale, so you can directly compare which feature has the most influence on the prediction.
3. Numerical Stability	Even for OLS (which doesn't use gradients), extreme value ranges can cause floating-point issues in the matrix math.

Comparing Coefficients After Normalization

With normalized Boston data, we can see that RM (average rooms, coeff = +19.17) has the strongest positive effect on price, while NOX (pollution, coeff = −7.84) and CRIM (crime, coeff = −7.25) have strong negative effects. Without normalization, these coefficients are on different scales and cannot be compared.

> **KEY POINT:** MinMax scaling is sensitive to outliers. A single extreme value can squish all other values near zero. Consider RobustScaler or StandardScaler if your data has heavy outliers.

Does Normalization Change OLS Accuracy?

For Scikit-Learn's LinearRegression (OLS), the MSE is virtually identical with or without normalization (both ≈21.65 train, 23.62 test). Normalization helps interpretability, not prediction quality, when using a direct matrix solver.

Part 7: What is Logistic Regression?

Why Linear Regression Fails at Classification

For classification (e.g., is this tumor malignant or benign?), the output must be 0 or 1. A straight line from linear regression produces values from −∞ to +∞, which is meaningless for class labels.

The Sigmoid Function

The sigmoid (or logistic) function squashes any real number into the range [0, 1]: `sigmoid(z) = 1 / (1 + e^(-z))`

Input z	sigmoid(z)	Interpretation
Very negative (e.g., −10)	≈ 0	Model is confident: class 0
Zero	0.5	Maximum uncertainty: 50/50
Very positive (e.g., +10)	≈ 1	Model is confident: class 1

Combining Linear + Sigmoid = Logistic Regression

Take the linear regression output and pass it through sigmoid: `P(y=1) = sigmoid(β0 + β1·x1 + … + βn·xn)`

Now the output is always between 0 and 1, interpreted as the probability of class 1. If P > 0.5, predict class 1; otherwise predict class 0.

> **Key Distinction**
>
> Despite its name, logistic regression is a CLASSIFICATION algorithm, not regression. The "regression" in the name refers to the fact that it still fits a linear equation internally — it just wraps the result in a sigmoid to produce probabilities.

Part 8: Logistic Regression in Practice (Breast Cancer)

The Dataset

The Wisconsin Breast Cancer dataset has 569 samples, each described by 30 features (radius, texture, perimeter, area, smoothness, etc.). The target is 0 (malignant) or 1 (benign).

Key Parameters of LogisticRegression

Parameter	Value	Meaning
C	100	Inverse regularization strength. Higher C = less regularization = model has more freedom to fit the data.
fit_intercept	True	Include the β0 bias term (almost always True).
solver	'lbfgs'	The optimization algorithm. LBFGS is a gradient-based method, which is why normalization matters here.
max_iter	100	Maximum gradient descent iterations. If the solver hasn't converged, it stops here.

Measuring Accuracy (Not MSE!)

For classification, we use **accuracy** — the percentage of correct predictions. The expression `np.mean(y == model.predict(X)) * 100` computes this by counting True values in a boolean array. Result: 99.3% training accuracy, 97.4% testing accuracy — excellent performance.

Impact of Normalization on Logistic Regression

Without normalization, the same model needs 100× more iterations (max_iter=10000 instead of 100) and still produces worse accuracy (97.8% train, 95.6% test vs 99.3% and 97.4%). This dramatically demonstrates why scaling is essential for gradient-based methods.

Part 9: Why Normalization Matters for Gradient-Based Solvers

The Gradient Update Rule

Gradient descent updates each weight with:

```
βi = βi − (2α/n) × Σ (ŷ_j − y_j) × x_ji
```

The Core Problem

Notice the gradient is **multiplied by the feature value x**. If feature x1 ranges 0–1 but feature x2 ranges 0–1000, then:

• The gradient for β2 is ~1000× larger than for β1

• A single learning rate α cannot work well for both: too small for β1 (learns painfully slowly) and too large for β2 (overshoots and oscillates)

> **Geometric Intuition**
>
> Un-normalized features create elongated elliptical loss surfaces. Gradient descent zigzags back and forth across the narrow dimension instead of heading straight toward the minimum. Normalizing makes the loss surface more circular, so gradient descent takes a direct path.

The Solution

Normalize all features to the same range (e.g., [0, 1] via MinMax). Then all gradients are on a similar scale, and a single learning rate works well for every weight simultaneously.

Part 10: Regularization — L1 and L2

What is Regularization?

When a model's learned weights become very large, it fits the training data too tightly (overfitting) and fails on new data. Regularization adds a penalty term to the loss function that discourages large weights, forcing the model to prefer simpler solutions.

Two Types of Regularization

Type	Penalty Term	Effect	Scikit-Learn Name
L1	$\lambda \times \Sigma\|w\|$	Pushes some weights to exactly zero — automatic feature selection	Lasso
L2	$\lambda \times \Sigma w^2$	Shrinks all weights toward zero but rarely makes any exactly zero	Ridge

λ (lambda) controls the penalty strength. Higher λ = stronger penalty = smaller weights. In Scikit-Learn's Ridge, this is the `alpha` parameter. In LogisticRegression, it is `C = 1/λ` (inverse relationship).

Seeing Regularization in Action

The lab creates a clever experiment: training data has slope = 3, but test data has slope = 2.5. Without regularization, the model learns slope ≈ 3 and overshoots the test data. With Ridge (alpha=10), the penalty shrinks the slope downward toward 2.5, coincidentally improving test performance.

> **KEY POINT:** This example works because the test slope happens to be lower than the training slope. Regularization always shrinks weights — if the true weights are already small, over-regularizing will hurt. You must use a validation set to choose the right alpha.

Part 11: Ridge Regression with Validation Set

The Three-Way Data Split

Instead of just train/test, we create three sets:

Set	Size	Purpose
Training	343 samples (68%)	Used to learn the model's weights.
Validation	61 samples (12%)	Used to tune hyperparameters (like alpha). Try many values, pick the one with lowest validation error.
Test	102 samples (20%)	Used ONCE at the very end to report final, unbiased performance. Never used for decisions.

Grid Search for the Best Alpha

We try alpha values [0, 0.01, 0.1, 1, 10, 100, 1000, 10000]. For each one, we train a Ridge model and evaluate it on the validation set (never the test set!).

Alpha	Validation MSE	Observation
0	19.88	Best — no regularization needed!
0.01	19.90	Negligible change
0.1	20.01	Slightly worse
1	21.20	Noticeably worse
10	31.17	Much worse
100	56.33	Severely over-regularized
1000	80.86	Model barely learns
10000	87.44	Almost predicting the mean

As alpha increases, the weights shrink more, the model becomes too simple, and the error climbs. For this dataset, regularization hurts — the best choice is alpha = 0 (equivalent to plain linear regression).

The Cost of Validation

Final model MSE (22.05 train, 24.37 test) is slightly worse than the earlier run without a validation split (21.65 train, 23.62 test). Why? We gave up 61 training samples to create the validation set. Less training data = a slightly weaker model. This is a real tradeoff in machine learning.

TIP: In practice, k-fold cross-validation can mitigate this tradeoff by reusing data more efficiently, but that is beyond this lab's scope.

Key Takeaways — Summary

Concept	What to Remember
Linear Regression	Finds the best straight line (or hyperplane) by minimizing squared errors. Scikit-Learn solves it exactly using matrix math (OLS).
Feature Engineering	Makes non-linear problems solvable by linear models. If $y \approx x^2$, just add x^2 as a feature column.
Train/Test Split	Essential for evaluating generalization. Never evaluate on training data alone.
Normalization	Puts all features on the same [0, 1] scale. Optional for OLS, but critical for gradient-based methods like logistic regression.
Logistic Regression	Linear regression + sigmoid function. Outputs probabilities in [0, 1] for binary classification.
Regularization (L1/L2)	Penalizes large weights to prevent overfitting. Ridge (L2) shrinks weights; Lasso (L1) can zero them out.
Validation Sets	Used to tune hyperparameters without touching the test set. Grid search over candidates, pick the lowest validation error.
Cost of Validation	Creating a validation set means less training data, which can slightly hurt final performance.